Advanced web programming project documentation

Declaration of AI usage:

1) Name all AI systems that were used in the development of the contents of this document, and for each:
   Claude
2) How and where they were used (illustrations, proofreading, getting ideas for text, to generate diagrams etc.):
   AI was used to check for grammar, proofread and for brainstorming ideas to what included in the document.

AWA Project is a web application that allows users to create, edit and share documents with other users through simple UI.

System and Architecture:

The frontend (React) communicates with backend (Node.js, Express). Authentication is handled through JWT tokens. MongoDB is used as database and Mongoose is used provide schema modeling.

Password hashing is done through bycrypt. Token and userID is store in the localStorage.

Schemas:

Document schema

```
interface IDocument extends Document {
    name: string
    content: string,
    date: Date,
    lastModified: Date,
    userID: string
    shared: string[]
    viewOnly: boolean,
    userEditing: string
}

const DocumentSchema: Schema = new Schema({
    name: {type: String, required: true},
    content: {type: String},
    date: {type: Date, default: Date.now},
    lastModified: {type: Date, default: Date.now},
    userID: { type: String, required: true },
    shared: [{ type: String}],
    viewOnly: { type: Boolean },
    userEditing: { type: String, default: null}
})
```

User schema

```
interface IUser extends Document {
    username: string
    password: string
}

const UserSchema: Schema = new Schema({
    username: {type: String, required: true, unique: true},
    password: {type: String, required: true}
})
```

Main features of the application:

Users can:

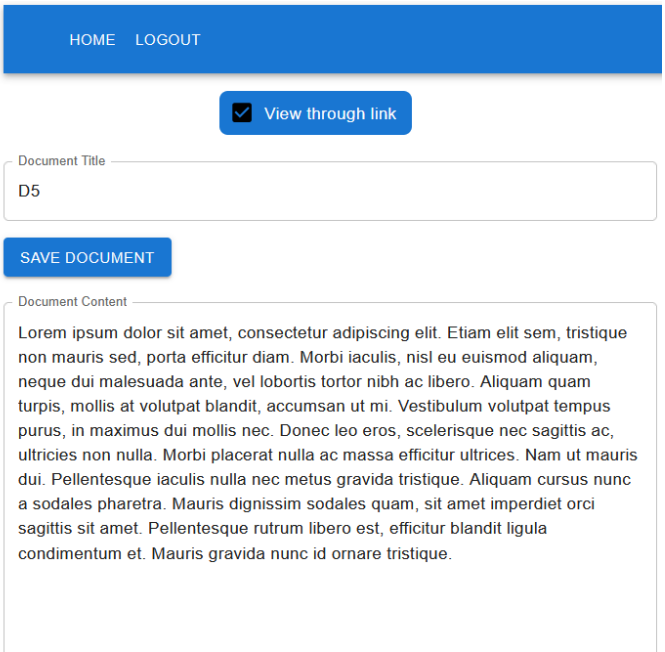
- Create, rename, edit and delete documents.
- Share with users through entering username to text field
- Share documents in view only mode by sharing the link to the document
- View when document was created and when was it last modified
- View documents other users have shared with them.

Users can create new documents by entering desired name of the document in the text field and then clicking the “CREATE DOCUMENT “-button. It is then displayed below on the table.

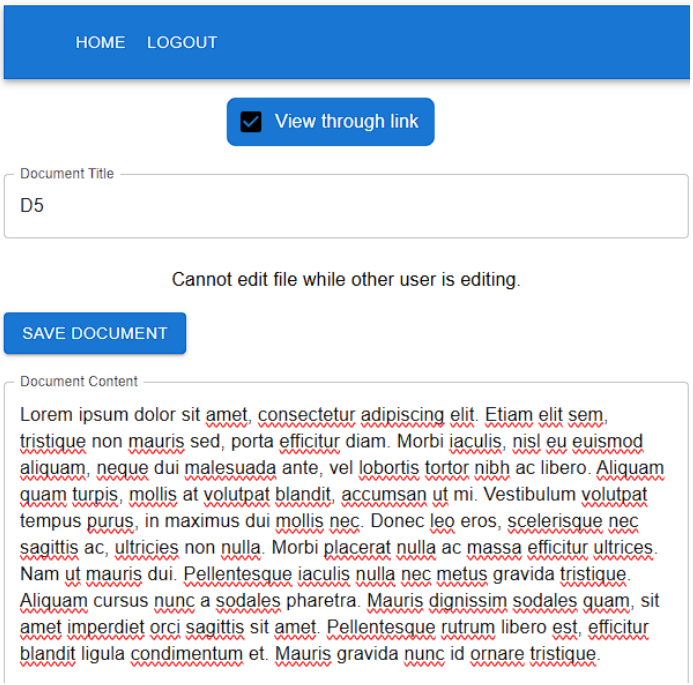
Users may edit the document only when no-one else is editing it. Once someone else is editing it the system displays simple user message to warn them that another user is editing it. Users may view the latest version of the document while another user is editing it. User may also set documents available to public through link. This can be achieved by checking the box “View through link” which enables anyone who has the link to view the document as they please. User may also disable availability this whenever they please.

User can browse through the files they have created from the home page. Documents that were created by the user are displayed first and documents that were shared to them is shown below that in its own table. In the tables it is shown the time when the document was created and when the last modification was made.

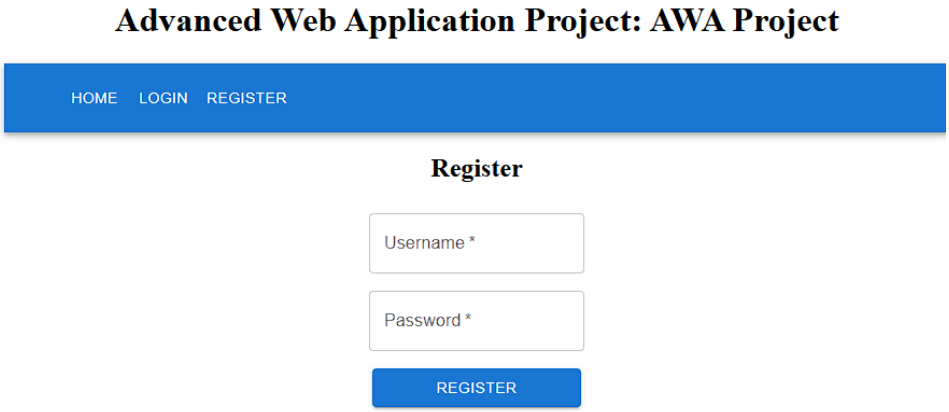
Below few pictures of the UI and some basic functions are shown such as the home page, login page and the registration page.



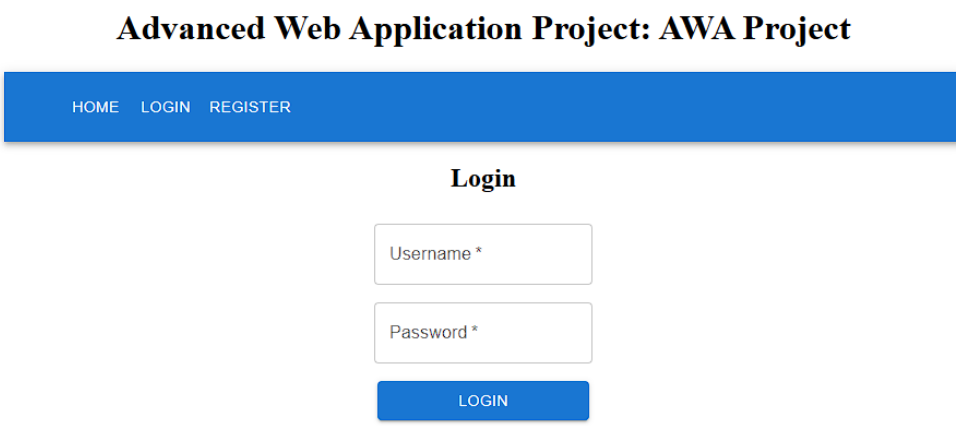
Picture 1. Normal view when editing document.



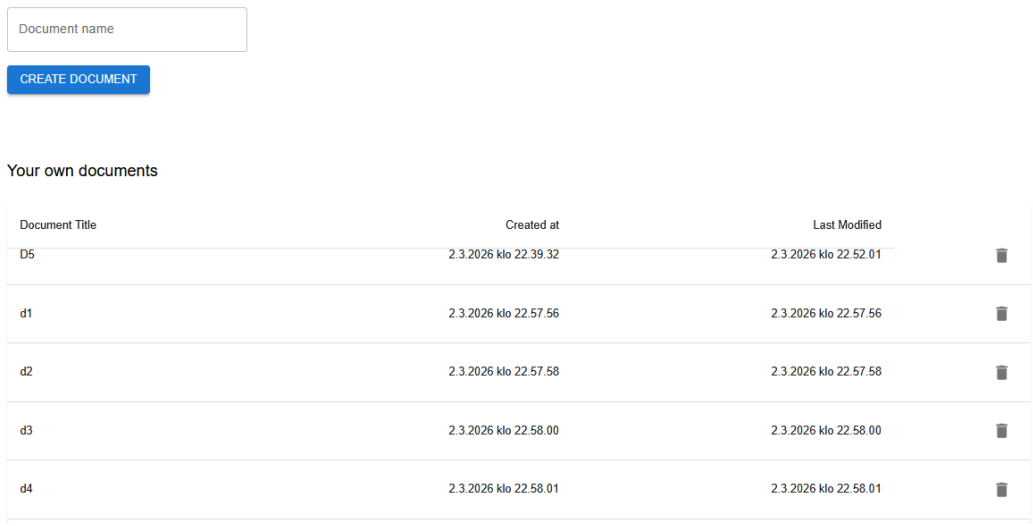
Picture 2. Error message once user tries to save document someone else is editing.



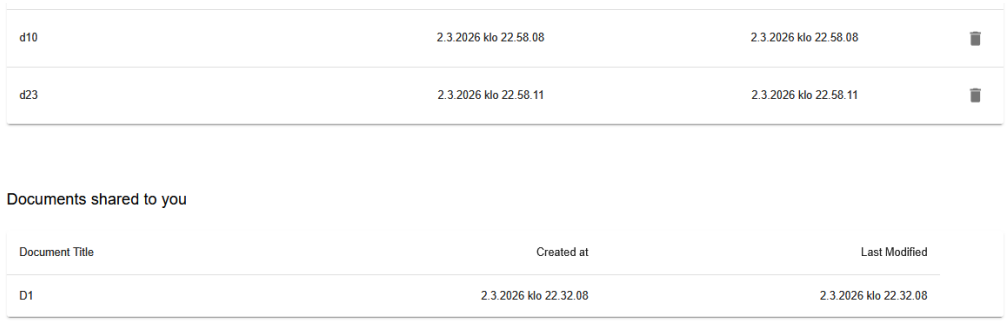
Picture 3. Registration page.



picture 4. Login page.



Picture 4. Main page with Your own documents -table.



Picture 5. Main page with Documents shared to you -table.

Document editing:

Main idea to restrict users from editing files others are currently editing was to restrict their access to the document page all together. However, I could not make the functionality work. It sometimes restricted view only users from viewing the document when someone was editing it or sometimes it just allowed multiple users to edit at the same time. Most of that system is still being worked on and they are still there but I opted for another system.

Currently when user tries to edit a file another is editing it does not allow them to save the document. Instead message is shown with a warning that document is already being edited.

Document may only be deleted by the user that created the file. The functionality has been removed from the users that the document was shared by the Share Document -function and that is by the design.

Technology used:

In this application the backend was done with Express.js and Node.js. Frontend was done with React, TypeScript and Material-UI. These technologies were chosen because I was already familiar with them through the course and new ideas to use them was mainly gotten from the technologies documentation such as “TableContainer” component which was imported through MaterialUI. MaterialUI was selected because it was easy to use and I was already a bit familiar with it.

For the database I used MongoDB and Mongoose.

Testing

Testing was done mainly manually. Each page and possible outcomes were tested by hand to see which functions work, and which do not work. The main issue and likely the only is that the restriction of users entering the document that is being edited does not work as I intend it to be.

Backend API calls:

document.ts

- "/addDocument" Creates new document returning new document object
- “/getDocument” Gets documents from the database
- “/getSharedDocument” Gets documents that were shared to user
- “/deleteDocument/:id” deletes selected document using userID
- “/OpenDocument/:id” Opens the selected document into the new page. Returns document data

- "/editDocument/:id" Saves the changes made by the user. Updates the document or returns error 400 when another user is editing
- "/shareDocument:id" Shares the document to another user by adding their ID to a array in the document data.
- "/editing/:id" Gets editing users ID and sets it as editingUser in the document data preventing others editing at the same time.
- "/releaseEditing/:id" Removes the users ID from the document data when exiting editing page.

<u>user.ts</u>

- "/register" Creates new user and hashes their password.
- "/login" Validates the user that is logging in and returns the userID and token.

<u>Points:</u>

Features	Points
Basic features and well written documentation	25
Frontside framework (React)	3
Own feature: Table to show user the files that were shared to them by other users. Adds more clarity for user and does not have to rely on links given by other users to get to the documents. Easy to brows already shared documents.	1-2
Total	29-30